Pipeline Description

Overview

The deployment pipeline is configured in `.circleci/config.yml`. CircleCI is connected to the GitHub repository and runs the workflow when code is pushed to the `main` or `master` branch.

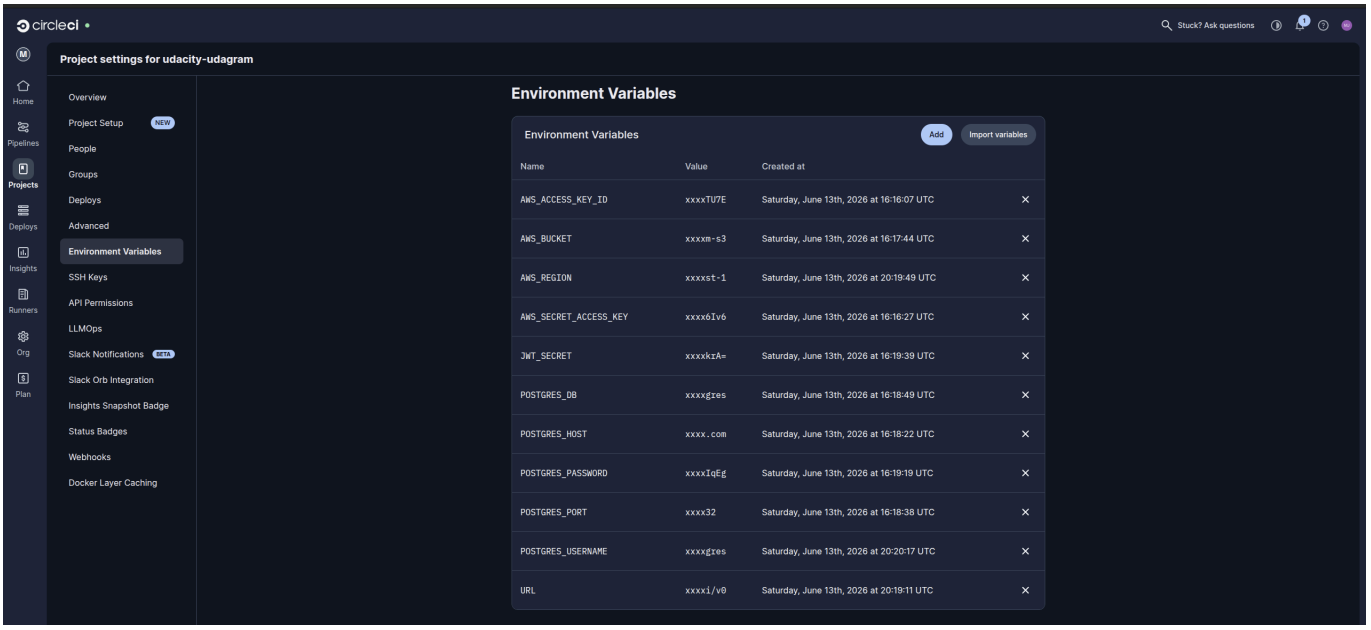
Pipeline Steps

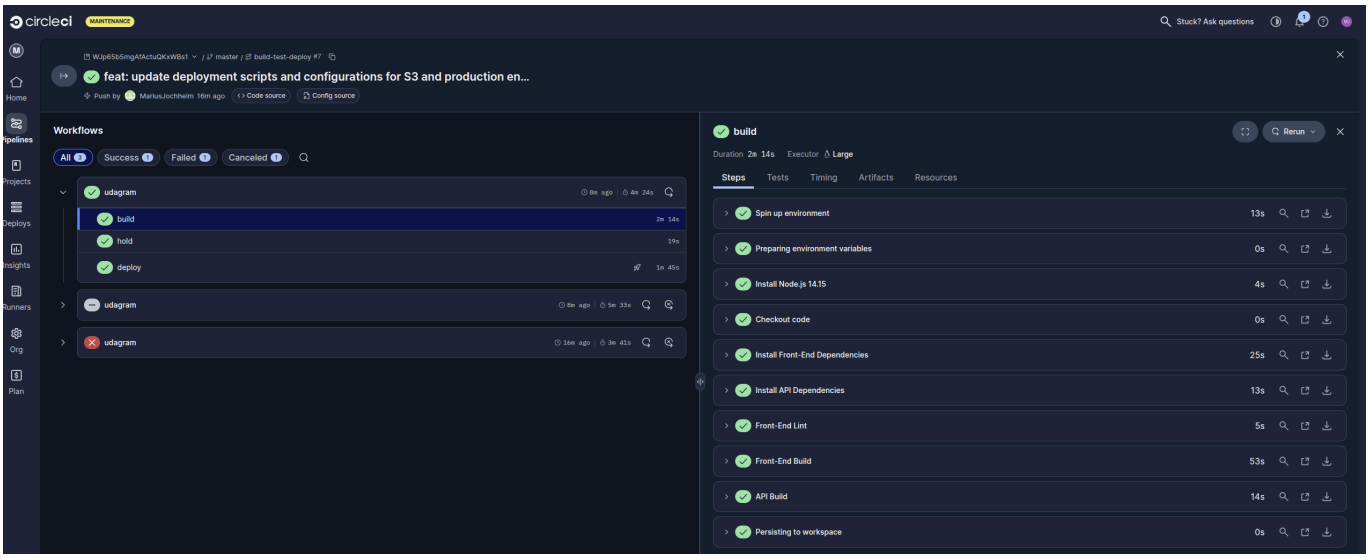
1. Checkout source code from GitHub.
2. Install frontend dependencies with `npm run frontend:install`.
3. Install API dependencies with `npm run api:install`.
4. Run frontend linting with `npm run frontend:lint`.
5. Build the frontend with `npm run frontend:build`.
6. Build the API with `npm run api:build`.
7. Persist the frontend and API build outputs to the CircleCI workspace.
8. Wait for manual approval in the `hold` job.
9. Deploy the API to Elastic Beanstalk with `npm run api:deploy`.
10. Deploy the frontend build to S3 with `aws s3 sync`.
11. Configure the S3 bucket for static website hosting.

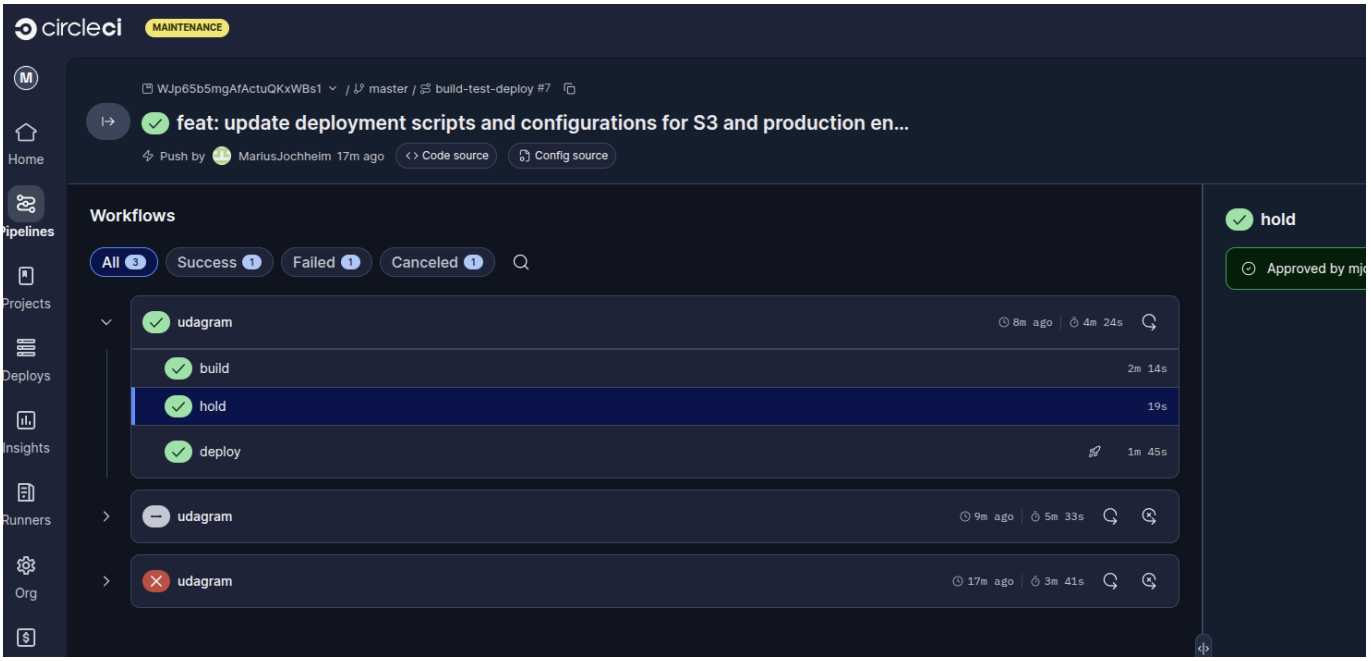
Pipeline Diagram

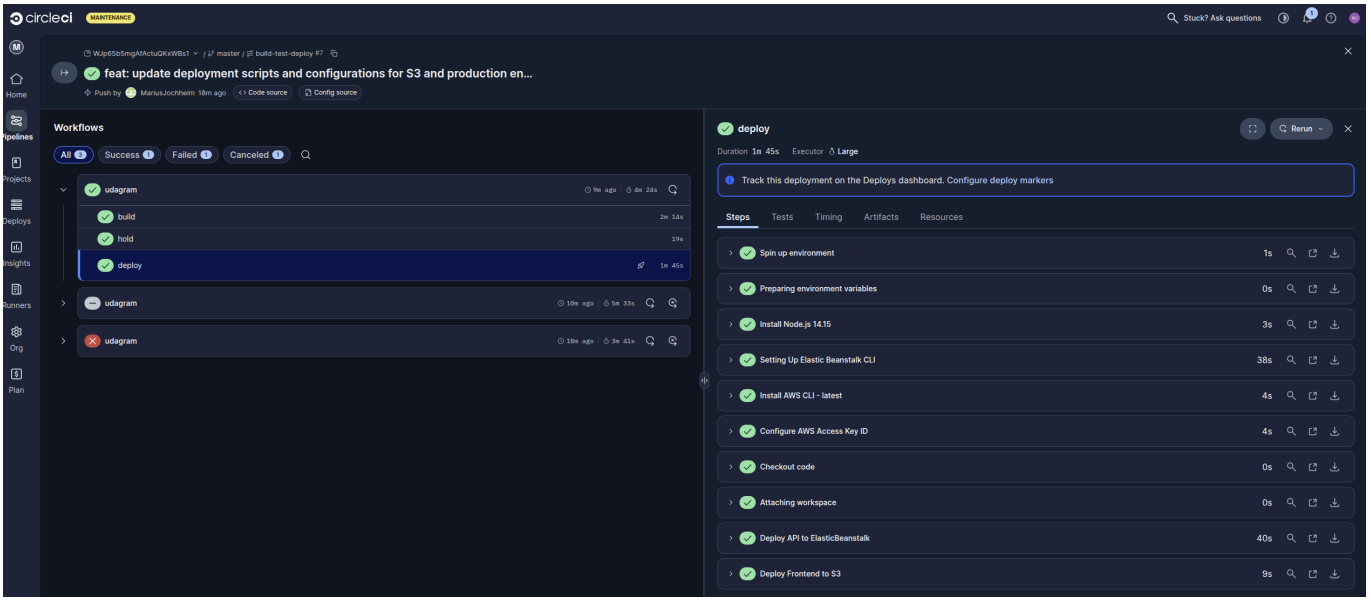
See Pipeline.md.

CircleCI Screenshots




circleci
MAINTENANCE
Stuck? Ask questions
feat: update deployment scripts and configurations for S3 and production en...
Workflows
All 3
Success 1
Failed 1
Canceled 1
udagram
build
hold
deploy
udagram
udagram
build
Rerun
Duration 2m 14s
Executor Large
Steps
Tests
Timing
Artifacts
Resources
Spin up environment 13s
Preparing environment variables 0s
Install Node.js 14.15 4s
Checkout code 0s
Install Front-End Dependencies 25s
Install API Dependencies 13s
Front-End Lint 5s
Front-End Build 53s
API Build 14s
Persisting to workspace 0s


circleci
MAINTENANCE
WJp65b5mgAfActuQKxWBs1 / master / build-test-deploy #7
feat: update deployment scripts and configurations for S3 and production en...
Push by MariusJochheim 17m ago
Code source
Config source
Workflows
All 3
Success 1
Failed 1
Canceled 1
udagram 8m ago 4m 24s
build 2m 14s
hold 19s
deploy 1m 45s
udagram 9m ago 5m 33s
udagram 17m ago 3m 41s
hold
Approved by mj


circleci
MAINTENANCE
Stuck? Ask questions
feat: update deployment scripts and configurations for S3 and production en...
Workflows
udagram
build
hold
deploy
udagram
udagram
deploy
Rerun
Duration 1m 45s
Executor Large
Track this deployment on the Deploys dashboard. Configure deploy markers
Steps
Tests
Timing
Artifacts
Resources
Spin up environment 1s
Preparing environment variables 0s
Install Node.js 14.15 3s
Setting Up Elastic Beanstalk CLI 38s
Install AWS CLI - latest 4s
Configure AWS Access Key ID 4s
Checkout code 0s
Attaching workspace 0s
Deploy API to ElasticBeanstalk 40s
Deploy Frontend to S3 9s

Secrets

Production secrets are configured outside the repository. CircleCI stores the required deployment and application secrets as project environment variables. During the deploy job, CircleCI validates that the required variables exist, uses the AWS credentials to deploy the backend bundle to Elastic Beanstalk, and runs `eb setenv` so the Elastic Beanstalk production environment receives the same runtime values that the backend reads from `udagram/udagram-api/src/config/config.ts`.

These values are not committed to the repository. The checked-in `udagram/set_env.sh` file is only a local development template.

Required CircleCI project environment variables:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `AWS_REGION`
- `AWS_BUCKET`
- `POSTGRES_HOST`
- `POSTGRES_DB`
- `POSTGRES_USERNAME`
- `POSTGRES_PASSWORD`
- `JWT_SECRET`
- `URL`

Do not use `POSTGRES_USER` or `AWS_DEFAULT_REGION` as the project variable names for this application. The backend configuration expects `POSTGRES_USERNAME` and `AWS_REGION`, so CircleCI and Elastic Beanstalk must use those exact names.

The CircleCI AWS and Elastic Beanstalk tooling expects `AWS_DEFAULT_REGION` internally. The deploy job derives that value from `AWS_REGION` at runtime, so `AWS_DEFAULT_REGION` does not need to be stored as a separate CircleCI project secret.

The deploy job passes these backend runtime values to Elastic Beanstalk:

```
eb setenv \
  POSTGRES_USERNAME="$POSTGRES_USERNAME" \
  POSTGRES_PASSWORD="$POSTGRES_PASSWORD" \
  POSTGRES_HOST="$POSTGRES_HOST" \
  POSTGRES_DB="$POSTGRES_DB" \
  JWT_SECRET="$JWT_SECRET" \
  AWS_BUCKET="$AWS_BUCKET" \
  AWS_REGION="$AWS_REGION" \
  AWS_ACCESS_KEY_ID="$AWS_ACCESS_KEY_ID" \
  AWS_SECRET_ACCESS_KEY="$AWS_SECRET_ACCESS_KEY" \
  URL="$URL"
```

After updating the CircleCI project settings, replace the CircleCI environment variables screenshot with a new screenshot showing the exact names above.